

PE6201 · Class 2 — RAG, the API version

The **same** RAG pipeline as `PE6201_Class2_RAG_local`, but the answer is generated by a **hosted, best-in-class model** you call over an API (and, where your provider supports it, embeddings too).

You need an API key. Paste it when prompted — it is read with `getpass`, so it is **never saved in the notebook**. Do not type your key into a code cell.

Cost / rate limits: every call costs tokens and providers rate-limit. Keep `TOP_K` small, run a few queries, and don't loop over the whole class at once.

RAG in one line: **embed your notes -> retrieve the ones nearest your question -> let a model answer from them.**

Setup

```
In [30]: %pip -q install openai sentence-transformers numpy
```

Note: you may need to restart the kernel to use updated packages.

Connect to your provider

Default is **OpenRouter** (one key, many models). The key is read from an **environment variable** — `MY_PRIVATE_OPENROUTER_KEY` (instructor) or `OPENROUTER_API_KEY` (students; or Colab **Secrets** ). If neither is set, you'll be prompted to paste it (hidden, never saved in the notebook). Same route as the Class 1 notebook.

```
In [31]: import os, getpass
from openai import OpenAI

def _get_openrouter_key():
    # 1) environment variable (same route as Class 1: instructor key, then student key)
    k = os.environ.get('MY_PRIVATE_OPENROUTER_KEY') or os.environ.get('OPENROUTER_API_KEY')
    if k: return k
    # 2) Google Colab Secrets: add OPENROUTER_API_KEY in the  panel
```

```

try:
    from google.colab import userdata
    k = userdata.get('OPENROUTER_API_KEY')
    if k: return k
except Exception:
    pass
# 3) last resort: paste it (hidden; never written into the notebook)
return getpass.getpass('Paste your OpenRouter API key (hidden): ')

API_KEY = _get_openrouter_key()
BASE_URL = 'https://openrouter.ai/api/v1' # OpenAI-compatible; change for another provider
client = OpenAI(base_url=BASE_URL, api_key=API_KEY)
print('OpenRouter key loaded.' if API_KEY else 'No key found – set the env var / Colab secret, or paste when prompt

# TRY THIS (knob): swap in any model your key can access.
# / हिन्दी via OpenRouter: 'qwen/qwen-2.5-7b-instruct', 'deepseek/deepseek-chat', 'google/gemini-flash-1.5' – ai
GEN_MODEL = 'openai/gpt-4o-mini' # already handles Chinese + Hindi well
# EMBED_MODEL – API embedding model (the provider must serve embeddings; OpenRouter often doesn't -> local fallback
# 'openai/text-embedding-3-small' -> 1536 dims (multilingual; default) . 'openai/text-embedding-3-large' -> 3
# other multilingual ( / हिन्दी) providers: Cohere 'embed-multilingual-v3.0' (1024), Voyage 'voyage-multilingual-
EMBED_MODEL = 'openai/text-embedding-3-small'

```

OpenRouter key loaded.

The knowledge base

Same eight Class 2 notes as the local notebook.

```

In [32]: # Our tiny knowledge base = 8 short notes from Class 2.
# In a real system this would be thousands of documents; the pipeline is identical.
DOCS = [
    "The modern AI stack has layers: compute, data and embeddings, a vector store, the model, orchestration, serving",
    "Retrieval-augmented generation (RAG) has three stages: embed your documents into vectors, retrieve the chunks",
    "Build-vs-buy is a decision per layer. Firms usually own their data and governance, rent frontier models, and m",
    "Training a model is a one-off cost; inference is paid on every request and usually dominates at scale. An H100",
    "Evaluation is a layer of the stack, not an afterthought. AI can fail silently: a fluent, confident answer that",
    "DBS, Singapore's largest bank, built its own data platform (ADA) and a responsible-data framework called PURE",
    "Fine-tuning bakes knowledge and behaviour into a model's weights and is good for a stable style or format. Ret",
    "Embeddings turn text into vectors so that similar meanings land near each other; retrieval finds the vectors n

```

```
]
print(f"{len(DOCS)} documents loaded")
```

8 documents loaded

Chunking — why we split (and overlap)

Before embedding, long documents are cut into **chunks**. Why not just embed a whole document?

- **Retrieval precision** — retrieval should return the relevant *passage*, not a 40-page manual. One vector per chunk means nearest-neighbour search finds the specific paragraph that answers the question.
- **One embedding = one meaning** — an embedding is a single point in meaning-space. Embed a long, multi-topic doc into one vector and you get a blurry *average* that matches everything a little and nothing well. Small chunks keep each vector about a single idea, so cosine similarity stays sharp.
- **Context window + cost** — the retrieved text is pasted into the prompt. Whole documents blow past the model's token limit, and every token costs money and latency — so we fetch a few small, on-target chunks.
- **Embedding input limits** — embedding models truncate long inputs anyway, silently dropping the tail.

Why overlap? Cut on a hard boundary and an idea that straddles two chunks gets split — each half is incomplete and neither retrieves as a full answer. Overlapping neighbours by a few words keeps a boundary idea whole in at least one chunk.

It's a **knob you tune against your evals**: too big → diluted, noisy vectors and wasted tokens; too small → each chunk loses the context needed to make sense. Turn `CHUNK_WORDS` / `OVERLAP` below and watch the split change.

```
In [33]: # TRY THIS (knob): chunk size (in words) and overlap between neighbours.
CHUNK_WORDS = 25      # try 12 (tiny chunks) or 200 (whole note = 1 chunk)
OVERLAP      = 10      # words shared with the next chunk, so an idea isn't cut in half

def chunk(text, size=CHUNK_WORDS, overlap=OVERLAP):
    words = text.split()
    if len(words) <= size:
        return [text]
    step = max(1, size - overlap)
    out = []
    for i in range(0, len(words), step):
        out.append(' '.join(words[i:i+size]))
        if i + size >= len(words):      # this chunk already reached the end
```

```

        break
    return out

# Build the chunks the retriever will use, from the knob above.
CHUNKS = [c for d in DOCS for c in chunk(d)]
print(f'{len(DOCS)} docs -> {len(CHUNKS)} chunks (CHUNK_WORDS={CHUNK_WORDS}, OVERLAP={OVERLAP})')

# See how each document is split, and the words neighbouring chunks SHARE (the overlap).
for d_i, doc in enumerate(DOCS, 1):
    cs = chunk(doc)
    print(f'\nDoc {d_i}: split into {len(cs)} chunk(s)')
    for c_i, c in enumerate(cs):
        print(f'    chunk {c_i} ({len(c.split())}w): {c}')
    if len(cs) > 1:
        shared = ' '.join(cs[0].split()[-OVERLAP:])
        print(f'    -> chunks 0 & 1 overlap by {OVERLAP} words: "{shared}"')

```

8 docs -> 17 chunks (CHUNK_WORDS=25, OVERLAP=10)

Doc 1: split into 2 chunk(s)

chunk 0 (25w): The modern AI stack has layers: compute, data and embeddings, a vector store, the model, orchestration, serving, and observability/evals. Each layer has a job, a

chunk 1 (15w): orchestration, serving, and observability/evals. Each layer has a job, a failure mode, and a cost.

-> chunks 0 & 1 overlap by 10 words: "orchestration, serving, and observability/evals. Each layer has a job, a"

Doc 2: split into 2 chunk(s)

chunk 0 (25w): Retrieval-augmented generation (RAG) has three stages: embed your documents into vectors, retrieve the chunks most similar to a question, and generate an answer from those

chunk 1 (19w): similar to a question, and generate an answer from those chunks. The model is rented; the retrieval is yours.

-> chunks 0 & 1 overlap by 10 words: "similar to a question, and generate an answer from those"

Doc 3: split into 2 chunk(s)

chunk 0 (25w): Build-vs-buy is a decision per layer. Firms usually own their data and governance, rent frontier models, and mix at the serving layer, judged on cost,

chunk 1 (16w): models, and mix at the serving layer, judged on cost, latency, control, data gravity, and regulation.

-> chunks 0 & 1 overlap by 10 words: "models, and mix at the serving layer, judged on cost,"

Doc 4: split into 3 chunk(s)

chunk 0 (25w): Training a model is a one-off cost; inference is paid on every request and usually dominates at scale. An H100 GPU rents for roughly 2

chunk 1 (25w): dominates at scale. An H100 GPU rents for roughly 2 to 7 US dollars per hour, and large models cost a few dollars per million

chunk 2 (11w): hour, and large models cost a few dollars per million tokens.

-> chunks 0 & 1 overlap by 10 words: "dominates at scale. An H100 GPU rents for roughly 2"

Doc 5: split into 2 chunk(s)

chunk 0 (25w): Evaluation is a layer of the stack, not an afterthought. AI can fail silently: a fluent, confident answer that is simply wrong. So you must

chunk 1 (21w): fluent, confident answer that is simply wrong. So you must measure quality with an eval set and monitor it in production.

-> chunks 0 & 1 overlap by 10 words: "fluent, confident answer that is simply wrong. So you must"

Doc 6: split into 2 chunk(s)

chunk 0 (25w): DBS, Singapore's largest bank, built its own data platform (ADA) and a responsible-data framework called PURE (Purposeful, Unsurprising, Respectful, Explainable), but it rents its foundation

chunk 1 (25w): PURE (Purposeful, Unsurprising, Respectful, Explainable), but it rents its foundation models. Its

```
-> chunks 0 & 1 overlap by 10 words: "PURE (Purposeful, Unsurprising, Respectful, Explainable), but it rents its foundation"
```

```
chunk 0 (25w): Fine-tuning bakes knowledge and behaviour into a model's weights and is good for a stable style or format. Retrieval looks facts up at query time
```

-> chunks 0 & 1 overlap by 10 words: "style or format. Retrieval looks facts up at query time"

chunk 0 (25w): Embeddings turn text into vectors so that similar meanings land near each other; retrieval finds the vectors nearest the question. Chunking and the number of

-> chunks 0 & 1 overlap by 10 words: "the vectors nearest the question. Chunking and the number of"

The default embedder is **English-only**, so Chinese / Hindi retrieval will be weak. To run this pipeline in another language: (1) paste one document set below into **DOCS** and re-run the corpus + chunking + embed cells, (2) pick a **multilingual embedder** in Stage 1, and (3) pick a **multilingual model** in Stage 3.

Chinese () – copy into DOCS :

Hindi (हिन्दी) — copy into DOCS :

```
DOCS = [
```

```
    "रिट्रीवल-ऑगमेंटेड जनरेशन (RAG) के तीन चरण हैं: दस्तावेज़ों को वेक्टर में बदलना, प्रश्न से मिलते-जुलते हिस्सों को खोजना, और उन्हीं हिस्सों से उत्तर तैयार करना। मॉडल किराये का है, रिट्रीवल आपका अपना।",
```

```
    "एम्बेडिंग टेक्स्ट को वेक्टर में बदलती है ताकि समान अर्थ वाले वाक्य पास-पास रहें; चंकिंग और top-k अक्सर मॉडल से ज़्यादा गुणवत्ता तय करते हैं।",
```

```
    "मूल्यांकन (evals) स्टैक की एक परत है, बाद की सोच नहीं: AI चुपचाप गलत हो सकता है, इसलिए गुणवत्ता मापना ज़रूरी है।",
```

```
]
```

Sample text for practice — swap in your own domain documents.

Stage 1 — Embed

We try the **API** embeddings first. If your provider doesn't serve embeddings (OpenRouter currently focuses on chat), the cell automatically falls back to a small **local** embedder — so the notebook always runs.

In [34]: `import numpy as np`

```
# TRY THIS (knob): if the API can't serve embeddings, we fall back to THIS local model.
# 'all-MiniLM-L6-v2' -> 384 dims (English, fast; default fallback)
# multilingual ( / हिन्दी): 'intfloat/multilingual-e5-base' (768) or 'BAAI/bge-m3' (1024)
LOCAL_EMBED_FALLBACK = 'all-MiniLM-L6-v2'
```

```
def _normalize(v):
    v = np.asarray(v, dtype='float32')
    return v / (np.linalg.norm(v, axis=-1, keepdims=True) + 1e-9)
```

```
_local_embedder = None
```

```
def embed(texts):
    global _local_embedder
    try:
        r = client.embeddings.create(model=EMBED_MODEL, input=list(texts))
        print(f'using API embeddings: {EMBED_MODEL}')
        return _normalize([d.embedding for d in r.data])
    except Exception as e:
        print('API embeddings unavailable ->', str(e)[:120])
        print(f'Falling back to local embeddings ({LOCAL_EMBED_FALLBACK}).')
        if _local_embedder is None:
            from sentence_transformers import SentenceTransformer
```

```

        _local_embedder = SentenceTransformer(LOCAL_EMBED_FALLBACK)
        return _normalize(_local_embedder.encode(list(texts)))

doc_vecs = embed(CHUNKS)
print(f'embedded {doc_vecs.shape[0]} chunks into {doc_vecs.shape[1]}-dim vectors')

```

using API embeddings: openai/text-embedding-3-small
 embedded 17 chunks into 1536-dim vectors

Embedding models differ — dimensionality, quality, speed

The size printed above is the **dimensionality**. Hosted embedders vary too — e.g. OpenAI `text-embedding-3-small` is **1536-dim** and `-3-large` is **3072-dim**; the local fallback (`all-MiniLM-L6-v2`) is **384-dim**.

- **More dimensions** $\approx$ more nuance, but more storage, slower search, higher cost.
- **Be consistent:** query and documents must use the *same* embedding model.
- **Chinese / Hindi:** OpenAI's `text-embedding-3-*` are multilingual (Chinese/Hindi OK). But the **local fallback** (`all-MiniLM-L6-v2`) is English-only — if you fall back, switch it to a multilingual model like `intfloat/multilingual-e5-base` or `BAAI/bge-m3`.
- **Cost:** API embeddings are billed per token — cheap ($\sim \$0.02$ / 1M) but not free at scale, and you re-embed when documents change.

Play later: swap `EMBED_MODEL` at the top between sizes/providers and re-run; if the API serves embeddings you'll see its dimension, otherwise the 384-dim local fallback.

Stage 2 — Retrieve

Identical to the local notebook — retrieval is the layer you own.

```

In [35]: import numpy as np

def cosine_top_k(query_vec, doc_vecs, k):
    # vectors are L2-normalised, so a dot product = cosine similarity
    sims = doc_vecs @ query_vec
    idx = np.argsort(-sims)[:k]
    return [(int(i), float(sims[i])) for i in idx]

```

```
# TRY THIS (knob): change how many chunks we retrieve.
TOP_K = 3
```

```
In [36]: def retrieve(query):
        q = embed([query])[0]
        hits = cosine_top_k(q, doc_vecs, TOP_K)
        return [CHUNKS[i] for i, _ in hits]

        # Demo: print the query, then the top-k chunks WITH their cosine similarity to it.
        demo_q = 'what is RAG?'
        print('QUERY:', demo_q)
        print(f'Top-{TOP_K} chunks by cosine similarity:')
        for i, sim in cosine_top_k(embed([demo_q])[0], doc_vecs, TOP_K):
            print(f'  sim={sim:.3f}  {CHUNKS[i]}')
```

QUERY: what is RAG?

Top-3 chunks by cosine similarity:

using API embeddings: openai/text-embedding-3-small

sim=0.462 Retrieval-augmented generation (RAG) has three stages: embed your documents into vectors, retrieve the chunks most similar to a question, and generate an answer from those

sim=0.426 the vectors nearest the question. Chunking and the number of retrieved chunks (top-k) usually decide RAG quality more than the model does.

sim=0.396 PURE (Purposeful, Unsurprising, Respectful, Explainable), but it rents its foundation models. Its DBS-GPT assistant pairs rented models with an internal knowledge base, which is RAG.

Stage 3 — Generate (via API)

This is the **one cell** that differs from the local notebook: instead of a small local model, we call a frontier model you rent by the token.

```
In [37]: def answer(query):
        context = '\n'.join(retrieve(query))
        prompt = (
            'Answer the question using ONLY the context below. '
            'If the answer is not in the context, say you do not know.\n\n'
            f'Context:\n{context}\n\nQuestion: {query}\nAnswer:'
        )
        resp = client.chat.completions.create(
            model=GEN_MODEL,
            messages=[{'role': 'user', 'content': prompt}],
```

```
        temperature=0,  
    )  
    return resp.choices[0].message.content
```

Try it

```
In [38]: # TRY THIS (knob): ask anything answerable from the notes above.  
# e.g. "What did DBS build versus buy?" · "Why do evals matter?" · "What costs the most?"  
QUERY = "What is RAG, in one sentence?"  
  
print(answer(QUERY))
```

using API embeddings: openai/text-embedding-3-small

RAG (Retrieval-augmented generation) is a process that involves embedding documents into vectors, retrieving the most similar chunks to a question, and generating an answer from those chunks.

How the model uses those chunks (the mechanics)

The `TOP_K` chunks aren't sent one at a time — they're concatenated into **one prompt** (instruction + all chunks + your question), and the model reads it in a single pass.

- **All chunks are in view.** A transformer uses **self-attention**, so while writing each word of the answer it can look at *every* token — all the retrieved chunks and your question. The answer is generated token by token, each conditioned on that whole context.
- **It doesn't average or vote.** The model leans on the chunk(s) that actually answer the question and mostly ignores the rest — and can *combine* facts across chunks when needed.
- **Irrelevant chunks aren't free.** They still cost tokens and can act as **distractors** — the noise side of the `TOP_K` knob (C3): too few misses the answer, too many dilutes it.
- **Position matters ("lost in the middle").** Models attend more to the start and end of the context than the middle, so a great chunk buried in the middle can be under-used — which is why **re-ranking** (best chunk first) helps (C3, *beyond naive RAG*).
- **"Use ONLY the context" is a strong bias, not a guarantee.** The model can still drift or invent — exactly why we measure **faithfulness / groundedness** in the evals section (C4).

Ties to the class: the model is the *rented* layer (C1); retrieval + chunking + top-k + re-ranking are the levers you *own* (C3); evals tell you whether the answer is actually grounded (C4).

Now experiment

You did not have to write RAG from scratch — you ran it. Now turn the knobs and watch the answer change:

1. **Change** `TOP_K` to 1, then 6 (retrieve cell), and `CHUNK_WORDS` to 15 (re-run from the chunking cell). These two knobs move retrieval more than the model does.
2. **Change** `QUERY` to something the notes *don't* cover (e.g. *"What is the capital of France?"*). Watch it either say it doesn't know, or make something up — that's a **silent failure**, and why we need evals.
3. **Break retrieval on purpose:** set `TOP_K = 0` or pass an unrelated query, and see the answer degrade even though nothing errors out.
4. **Compare the two notebooks:** run the same `QUERY` here and in the *other* RAG notebook. Same pipeline — only the embed and generate cells differ (local vs API). That difference *is* build-vs-buy at the model layer.

Evaluate it — the seed of A1

You turned the knobs by eye. Now **measure** it — AI fails silently. Two levels (from C4):

- **L1 — assertions:** deterministic true/false checks in code (is an out-of-scope question refused? does the answer contain the expected fact? is it short enough?).
- **L2 — a judge:** for what you can't assert, ask a model (or a human) to score. Here: **faithfulness** — is the answer supported by the retrieved chunks?

This tiny harness is the exact shape of **A1**. (On the small local model some checks may fail — that's the point: measure, don't assume.)

How the L1 checks work (and how to design your own)

L1 = **assertions** — each check is a plain true/false test in code. There are two kinds of cases:

- **In-scope** (`must_include`) — the answer *must contain* an expected fact (we check for a keyword). PASS = the fact is present.
- **Out-of-scope** (`must_not_include`) — the model *must refuse* ("I don't know...") **and must not leak** the real-world answer. PASS = refused **and** did not leak.

Why “refuse AND no leak”? A refusal-phrase check *alone* gives a **false PASS** on a hedging answer like “I cannot find it in the context, but the capital of France is Paris.” — it contains a refusal phrase yet still leaks *Paris*. Requiring `not leaked` catches that.

Why we print the answer. A PASS you can’t see is a PASS you can’t trust. Printing the model’s answer under each question is the core habit — *read your data* — so you can confirm the check is right (and spot brittle ones).

Heuristics are brittle — both ways: a stray keyword can *false-PASS* a wrong answer; a valid refusal you didn’t list (e.g. “the context doesn’t mention that”) can *false-FAIL*. That brittleness is exactly why open-ended quality goes to **L2** (an LLM-as-judge) — keep L1 for the crisp, checkable things.

Designing yours (A1): for each case decide what a good answer must **contain** (`must_include`), must **avoid** (`must_not_include`), and any **format** bounds (length, valid JSON, no PII). Aim for ~10 cases in your own domain.

```
In [39]: # A tiny labelled eval set: each case knows what a good answer looks like.
EVALSET = [
    {'q': 'What is RAG, in one sentence?', 'in_scope': True, 'must_include': ['retriev']},
    {'q': 'What did DBS build versus buy?', 'in_scope': True, 'must_include': ['ada']},
    {'q': 'What usually decides RAG quality?', 'in_scope': True, 'must_include': ['chunk']},
    {'q': 'What is the capital of France?', 'in_scope': False, 'must_not_include': ['paris']},
]

REFUSALS = ['do not know', "don't know", 'not in the context', 'no information', 'cannot find', 'not mentioned', 'n

def l1_checks(case, ans):
    a = ans.lower()
    checks = {}
    if case['in_scope']:
        checks['mentions expected fact'] = all(k in a for k in case.get('must_include', []))
    else:
        refused = any(r in a for r in REFUSALS)
        leaked = any(bad in a for bad in case.get('must_not_include', []))
        # out-of-scope PASSES only if it refuses AND does not leak the real-world answer
        checks['refuses & no leak'] = refused and not leaked
    checks['answer <= 400 chars'] = len(ans) <= 400
    return checks

passed = total = 0
for case in EVALSET:
```

```

ans = answer(case['q'])
print(f"\nQ: {case['q']}")
print(f"A: {ans.strip()}")          # print the answer so every PASS/FAIL is verifiable
for name, ok in l1_checks(case, ans).items():
    total += 1; passed += int(ok)
    print((' PASS' if ok else ' FAIL'), '|', name)
print(f"\nL1 pass-rate: {passed}/{total} = {passed/total:.0%}")

```

using API embeddings: openai/text-embedding-3-small

Q: What is RAG, in one sentence?

A: RAG (Retrieval-augmented generation) is a process that involves embedding documents into vectors, retrieving the most similar chunks to a question, and generating an answer from those chunks.

PASS | mentions expected fact

PASS | answer <= 400 chars

using API embeddings: openai/text-embedding-3-small

Q: What did DBS build versus buy?

A: DBS built its own data platform (ADA) and a responsible-data framework called PURE, but it rents its foundation models.

PASS | mentions expected fact

PASS | answer <= 400 chars

using API embeddings: openai/text-embedding-3-small

Q: What usually decides RAG quality?

A: Chunking and the number of retrieved chunks (top-k) usually decide RAG quality more than the model does.

PASS | mentions expected fact

PASS | answer <= 400 chars

using API embeddings: openai/text-embedding-3-small

Q: What is the capital of France?

A: I do not know.

PASS | refuses & no leak

PASS | answer <= 400 chars

L1 pass-rate: 8/8 = 100%

In [40]: *# L2 - a real LLM-as-judge for faithfulness (is the answer supported by the context?).*

```

def judge_faithful(context, ans):
    prompt = (
        'You are grading an answer. Reply with only YES or NO.\n'

```

```

        'Is EVERY claim in the ANSWER supported by the CONTEXT?\n\n'
        f'CONTEXT:\n{context}\n\nANSWER:\n{ans}\n\nSupported (YES/NO):'
    )
    r = client.chat.completions.create(model=GEN_MODEL,
                                       messages=[{'role': 'user', 'content': prompt}], temperature=0)
    return r.choices[0].message.content.strip().upper().startswith('YES')

faithful = 0
for case in EVALSET:
    ctx = '\n'.join(retrieve(case['q']))
    ans = answer(case['q'])
    ok = judge_faithful(ctx, ans)
    faithful += int(ok)
    print(('FAITHFUL' if ok else 'UNSUPPORTED'), '|', case['q'][:40])
print(f'\nL2 (faithfulness) pass-rate: {faithful}/{len(EVALSET)} = {faithful/len(EVALSET):.0%}')
# Align your judge to a few human labels before trusting it (C4).

```

```

using API embeddings: openai/text-embedding-3-small
using API embeddings: openai/text-embedding-3-small
FAITHFUL | What is RAG, in one sentence?
using API embeddings: openai/text-embedding-3-small
using API embeddings: openai/text-embedding-3-small
FAITHFUL | What did DBS build versus buy?
using API embeddings: openai/text-embedding-3-small
using API embeddings: openai/text-embedding-3-small
FAITHFUL | What usually decides RAG quality?
using API embeddings: openai/text-embedding-3-small
using API embeddings: openai/text-embedding-3-small
UNSUPPORTED | What is the capital of France?

```

L2 (faithfulness) pass-rate: 3/4 = 75%

This is A1

A1 asks for exactly this, scaled up: **~10 cases** for a task in your industry, with **L1 assertions** (like above) **plus at least one L2 judge check**. Keep the harness runnable — the deliverable is the working checks, not an essay about evals.

What you just built

The **same RAG pipeline**, but the generate step (and, where available, embeddings) calls a **hosted API** — a frontier model you rent by the token.

Compare with **PE6201_Class2_RAG_local** : the pipeline is identical; only two cells changed. Better answers here, but you pay per request and depend on a provider. That is exactly the **build-vs-buy** decision at the model layer.